

Inferencia.

Ingeniería aplicada · Modelos de pesos abiertos · Harness

Angel Galvis

Founder and Managing Partner at Datastrat.

a@datastrat.co

DEFINICIÓN

Ingeniería de inferencia.

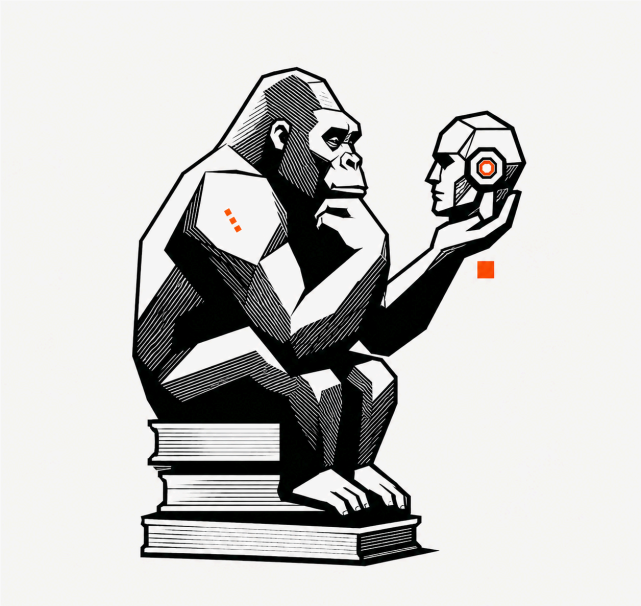
La ingeniería de inferencia es la disciplina dedicada al diseño, la operación y la optimización de sistemas que ejecutan modelos generativos en producción para atender solicitudes y producir resultados.

Su dimensión estratégica consiste en **escoger, entre las alternativas disponibles, una configuración coherente del sistema**, hacer explícitos los trade-offs entre rendimiento, eficiencia, disponibilidad y confiabilidad, y alinear cada elección con las necesidades del producto.

El producto **[negocio]** define la estrategia de inferencia.

Agenda.

01	Aplicación	¿Qué necesita realmente el producto?
02	Selección y medición	¿Qué modelo satisface esa necesidad y cómo se prueba?
03	Mecánica del modelo	¿Qué ocurre durante una inferencia?
04	Técnicas	¿Qué cuello de botella resuelve cada familia de optimización?
05	Cuantización aplicada	¿Qué se gana, qué se arriesga y qué hizo Alchemist?
06	Harness y composición	¿Cómo se convierte capacidad en un sistema operable y modular?



PRESENTACIÓN
HTML · PDF



NOTEBOOK
Kaggle



ALCHEMIST
Hugging Face

MARCO CONCEPTUAL

Agente.

Un agente ejecuta un ciclo iterativo.

El modelo interpreta el estado de una tarea y propone una acción. El harness valida y ejecuta la acción, incorpora el resultado al estado y determina si el proceso continúa, se recupera o termina.

Cada iteración modifica el contexto disponible para la siguiente decisión.



FUENTE [Trivedy \(2026\)](#); definición operativa adaptada por Datastrat.

MARCO CONCEPTUAL

Agente = Modelo + Harness.

MODELO

- interpreta
- relaciona
- propone
- opera de forma probabilística



HARNESS

- contexto y estado
- tools · skills · permisos
- políticas y validación
- ejecución y verificación

AGENTE → trayectoria observable

Trivedy, V. (2026). The anatomy of an agent harness. LangChain.
Hashimoto, M. (2026). My AI adoption journey.

MARCO CONCEPTUAL

Componentes de la ingeniería de inferencia aplicada.

Una buena estrategia de inferencia concentra el esfuerzo de ingeniería en dos frentes coordinados: la ejecución del modelo y la ingeniería del harness.

EJECUCIÓN DEL MODELO

pesos · precisión · software de inferencia · hardware · memoria

propuesta probabilística

INGENIERÍA DEL HARNESS

contexto · tools · skills · permisos · validación

control determinista

El modelo concentra la interpretación semántica y genera propuestas mediante un proceso probabilístico. El harness aplica mediante código reglas estables, contratos, cálculos exactos y condiciones de aceptación.

El sistema completo se evalúa extremo a extremo.

FUENTES [Hashimoto \(2026\)](#); [Trivedy \(2026\)](#).

MARCO CONCEPTUAL

Entrenamiento e inferencia.

ENTRENAMIENTO

Proceso de aprender los pesos del modelo a partir de datos.

INFERENCIA

Operación de un modelo generativo en producción para procesar entradas y producir resultados.

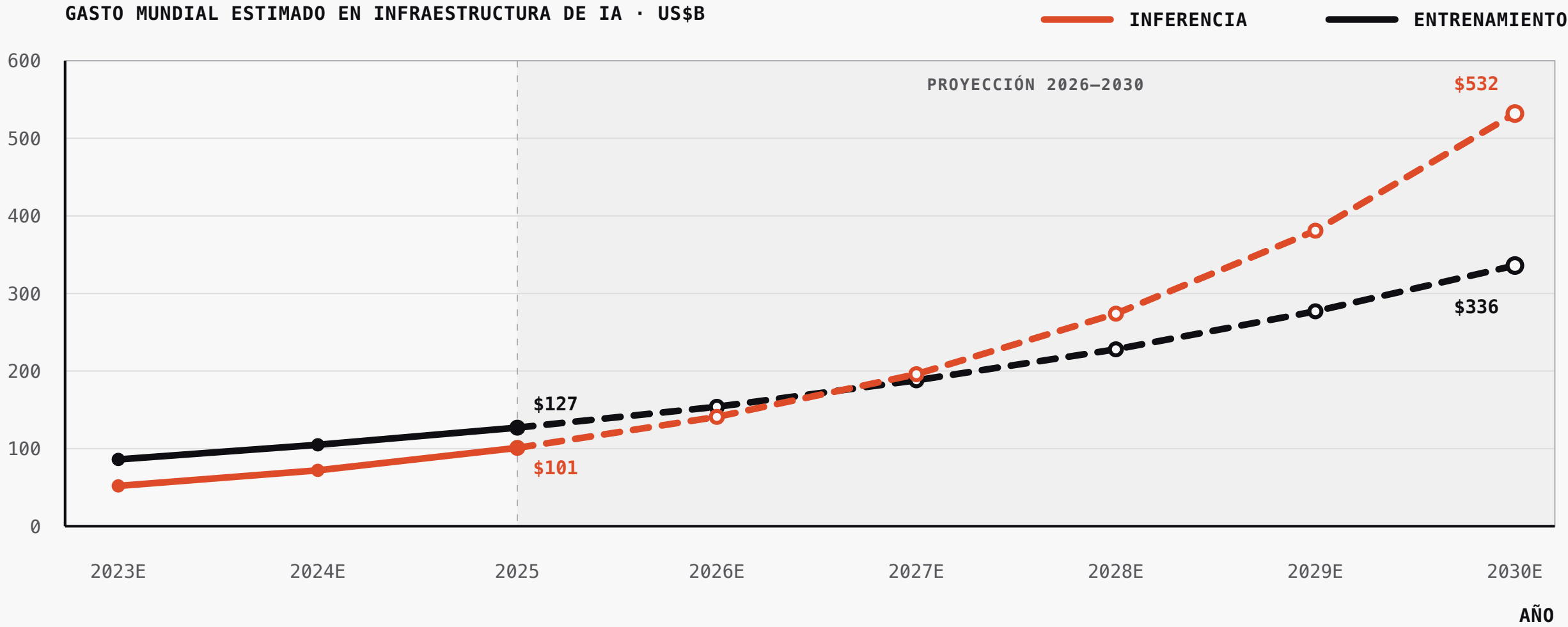
El entrenamiento crea o modifica la capacidad. La inferencia ocurre cada vez que se utiliza el producto y concentra su costo, latencia y confiabilidad operativa.



cada uso materializa **costo · latencia · confiabilidad**

MARCO CONCEPTUAL

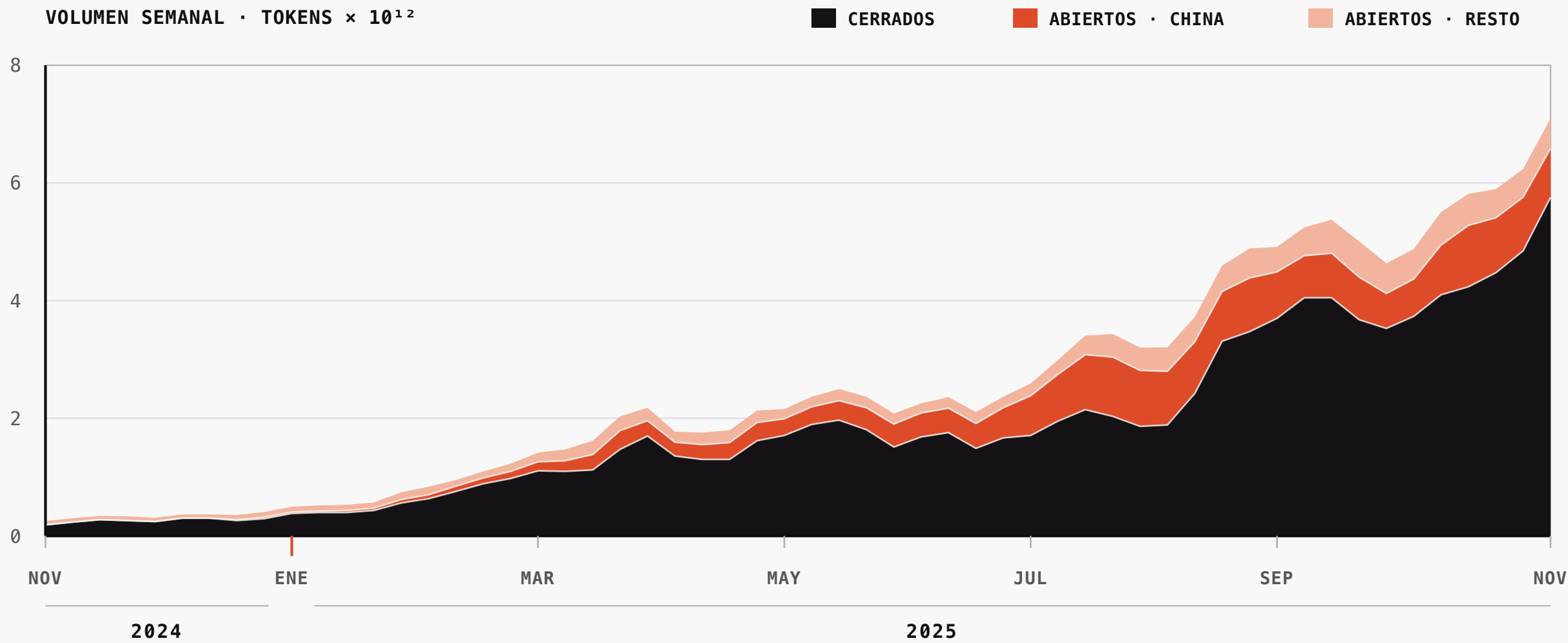
Entrenamiento e inferencia.



FUENTE Macatee / S&P Global Market Intelligence (2026). 2025 y 2030: publicados; resto: estimado con CAGR por carga. Mundial, incluida China; excluye preparación de datos. US\$B = miles de millones.

MARCO CONCEPTUAL

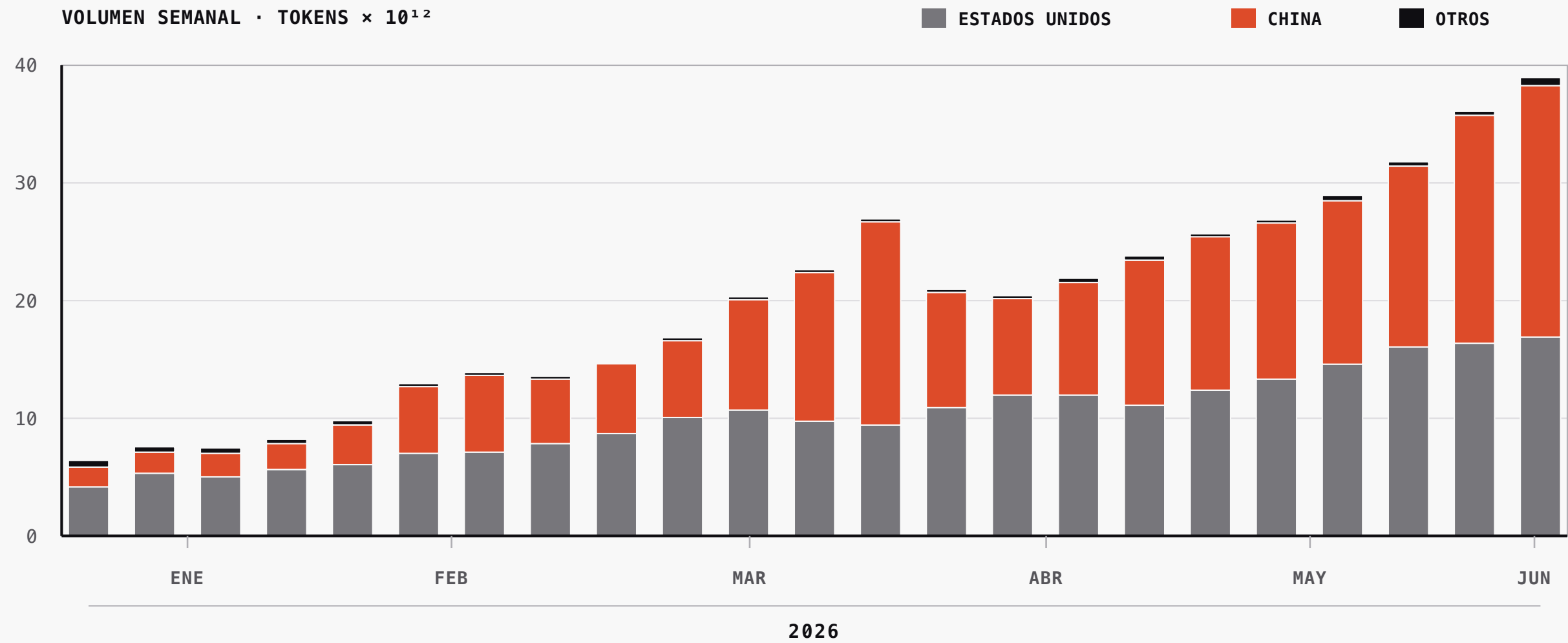
Tokens procesados por OpenRouter.



FUENTE [Aubakirova y Midha / OpenRouter-a16z \(2025\)](#). Tokens = entrada + salida; tráfico de OpenRouter, no mercado mundial. Valores semanales reconstruidos de la figura oficial.

MARCO CONCEPTUAL

Tokens procesados por OpenRouter según país de origen · 2026.



FUENTE [OpenRouter \(2026\)](#). Tokens = entrada + salida; país del autor del modelo. Valores semanales reconstruidos de la figura oficial.

01

Aplicación.

¿Qué necesita realmente el producto?

01 · APLICACIÓN

Requisitos de la aplicación.

Cada categoría de aplicación produce una carga distinta.

- Agentes: una acción puede generar varias inferencias.
- Conversación: el tiempo hasta el primer token determina la rapidez percibida.
- Voz: domina la latencia extremo a extremo.
- Búsqueda y moderación: pueden privilegiar rendimiento y costo.

MODALIDAD DE LA CARGA

- **En línea:** una persona o sistema espera la respuesta.
- **Fuera de línea:** admite mayor demora individual para procesar más trabajo.

¿QUÉ DEBE HACER LA APLICACIÓN?

TAREA	MODALIDAD	TRÁFICO	DATOS	SERVICIO
conversación · agente	texto · voz · lote	conurrencia · variabilidad	privacidad · residencia	latencia · disponibilidad

modelo · precisión · software de inferencia · hardware · despliegue · métricas

TIPO DE PRODUCTO

- **Consumo:** demanda más variable y mayor sensibilidad al costo por solicitud.
- **Empresas:** mayor exigencia de disponibilidad, latencia consistente y cumplimiento normativo.

01 · APLICACIÓN

Inferencia compartida y capacidad dedicada.

La inferencia compartida reduce el trabajo inicial: se paga por consumo, no requiere gestionar capacidad y el modelo permanece disponible. A cambio, el costo crece con el uso y el proveedor limita versión, latencia, capacidad y disponibilidad.

La capacidad dedicada añade gasto fijo y trabajo operativo, pero permite controlar la versión y ajustar el despliegue.

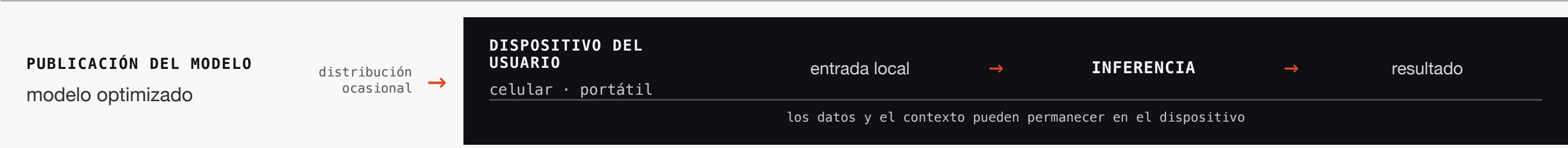
La transición se justifica por escala, especialización u orquestación y requiere una necesidad de negocio inmediata.

INFERENCIA COMPARTIDA	
VENTAJAS	DESVENTAJAS
Pago únicamente por consumo	El costo crece con el volumen de uso
El proveedor mantiene el modelo disponible	La disponibilidad del producto depende del proveedor
No requiere administrar GPUs ni capacidad	La infraestructura compartida puede introducir variabilidad
Integración inicial sencilla mediante una API	Menor control sobre versión, latencia, calidad y límites de uso
CAPACIDAD DEDICADA CUANDO escala · especialización · orquestación	

01 · APLICACIÓN

Inferencia local [edge].

La inferencia local —también llamada *edge*, *client-side* u *on-device*— ejecuta el modelo directamente en el dispositivo del usuario, en lugar de enviar cada solicitud a un servidor central.



LO QUE HABILITA

- **Menor latencia de red:** elimina el viaje de ida y vuelta al servidor.
- **Operación sin conexión:** puede funcionar después de distribuir el modelo.
- **Privacidad:** los datos sensibles pueden permanecer en el dispositivo.
- **Nueva estructura de costos:** el cómputo deja de concentrarse en el datacenter.

LO QUE LA CONDICIONA

- **Capacidad:** memoria y cómputo considerablemente más limitados.
- **Temperatura y batería:** la inferencia sostenida consume energía y produce calor.
- **Fragmentación:** múltiples combinaciones de hardware, sistema operativo y runtime.
- **Operación distribuida:** actualizar, observar y soportar muchos dispositivos.

Una arquitectura híbrida puede ejecutar en el dispositivo las tareas frecuentes o sensibles y reservar la nube para modelos mayores o cargas intensivas.

01 · APLICACIÓN

Runtime, infraestructura y herramientas.

RUNTIME

Optimiza el rendimiento de un modelo en una instancia con una o varias GPU, unidades de procesamiento gráfico.

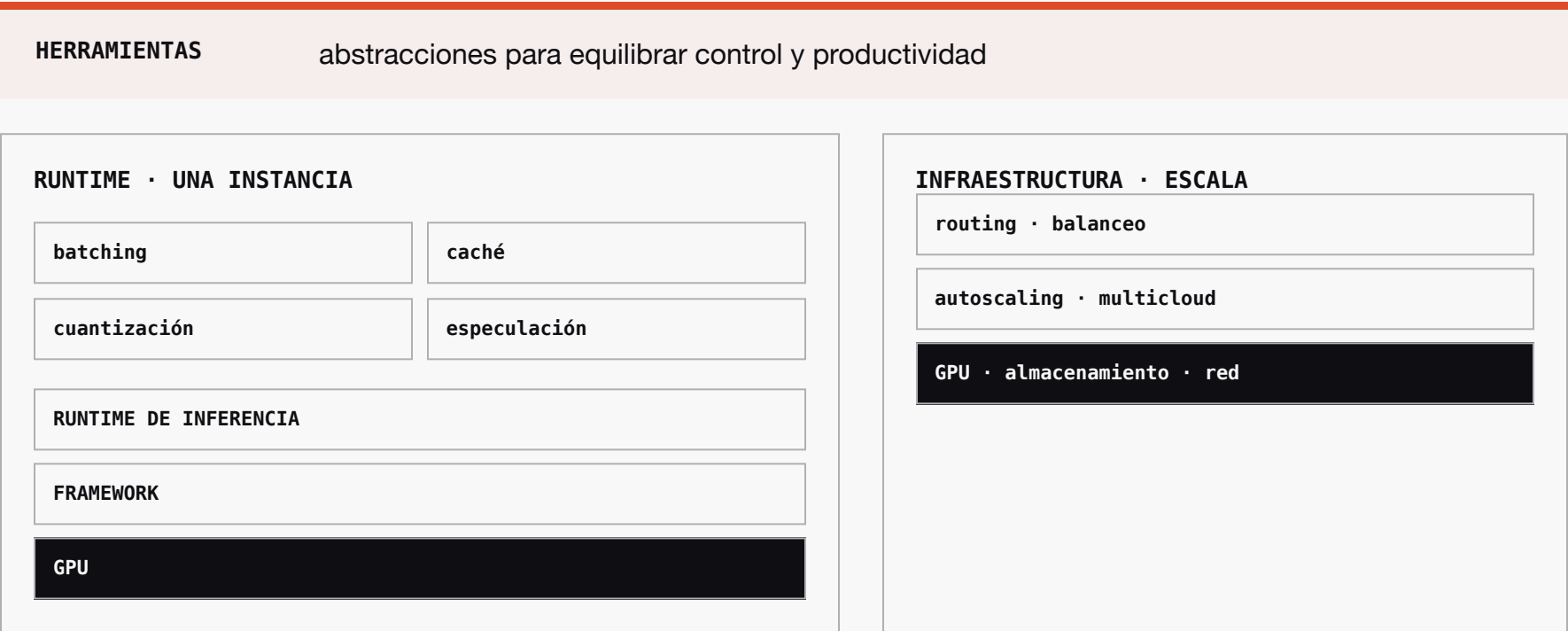
INFRAESTRUCTURA

Escala capacidad entre clústeres, regiones y nubes sin crear si-los, mientras mantiene la disponibilidad.

HERRAMIENTAS

Proporcionan a los equipos el nivel de abstracción necesario para equilibrar control y productividad.

Las tres capas deben funcionar juntas para operar inferencia crítica a escala.



02

Selección y medición.

¿Qué modelo satisface esa necesidad y cómo se prueba?

02 · SELECCIÓN Y MEDICIÓN

Acceso al modelo.

SERVICIO ADMINISTRADO

Acceso al modelo mediante una API.

El proveedor conserva los pesos y controla la versión y la infraestructura de ejecución.

PESOS ABIERTOS

Acceso a los parámetros conforme a su licencia.

El equipo puede escoger software de inferencia, precisión, hardware y forma de despliegue.

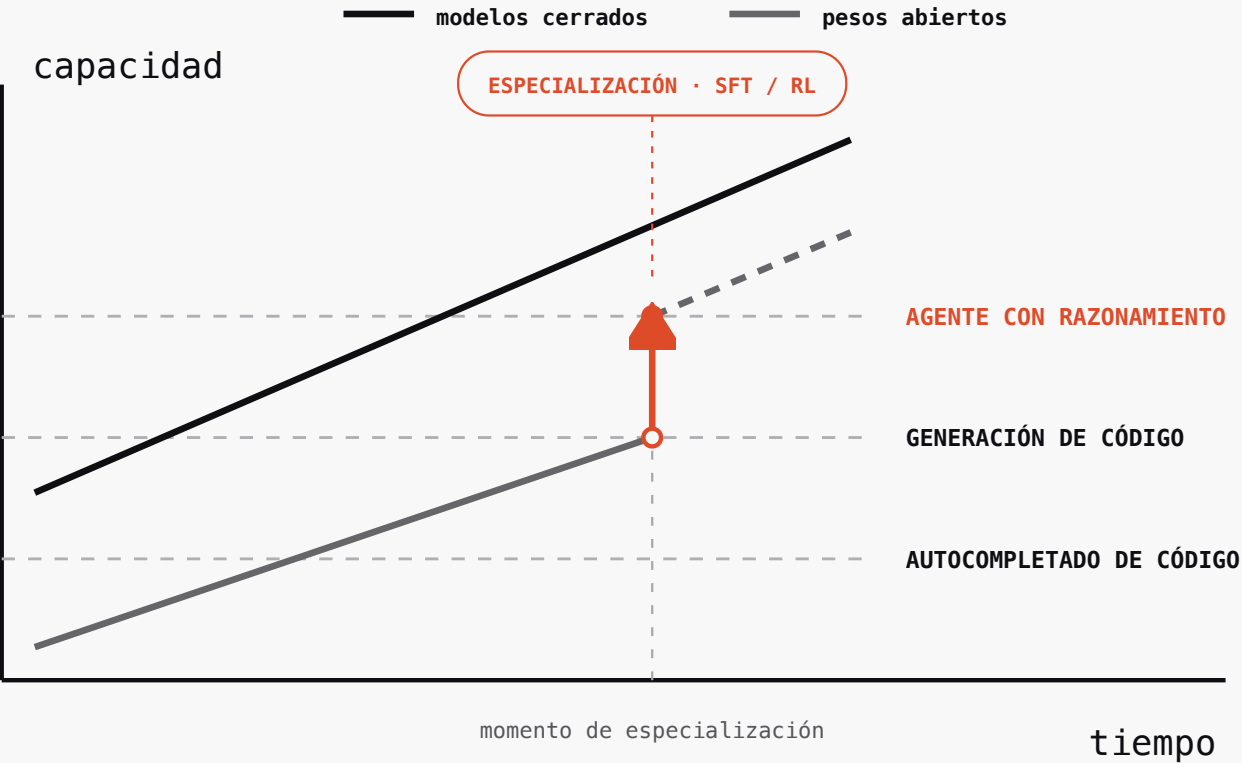
02 · SELECCIÓN Y MEDICIÓN

Modelos de pesos abiertos.

Un modelo de pesos abiertos publica sus parámetros para que puedan examinarse, ejecutarse o transformarse conforme a su licencia.

El acceso a los pesos no implica acceso a los datos de entrenamiento ni al proceso completo que los produjo.

El ajuste supervisado (SFT) o el aprendizaje por refuerzo (RL) pueden desplazar un modelo de pesos abiertos a una trayectoria de capacidad superior para una tarea concreta. Ese cambio puede permitir cruzar antes un umbral útil y conservar control sobre latencia, disponibilidad y economía.



FUENTE [Open Source Initiative \(2024\)](#).

Acceso como servicio y acceso a los pesos.



02 · SELECCIÓN Y MEDICIÓN

Tendencias del ecosistema.

SELF-HOSTED INTELLIGENCE

El equipo desacopla modelo, proveedor e infraestructura para controlar costo, rendimiento y datos.

	pesos abiertos	→	ajuste o enrutamiento	→	infraestructura elegida
UBER · 2026	Llama y Mixtral ajustados internamente · rendimiento similar a GPT-4 con mayor volumen.				
BLOCK · 2025	Goose local y multimodelo · 50–75 % menos tiempo de desarrollo.				
COINBASE · 2026†	Kimi y GLM dentro de una estrategia que redujo ≈50 % el gasto de IA.				
DOORDASH · 2026†	Kimi para tareas de menor nivel · mejor calidad a menor costo.				
PERPLEXITY · 2025	Sonar sobre Llama 3.3 70B · hasta 1.200 tokens por segundo.				

† Publicaron la elección de modelos, no el esquema de alojamiento.

INTELIGENCIA DE DOMINIO PROPIA

Modelos fundacionales entrenados sobre datos financieros propios y reutilizados en varios productos.

	transacciones	→	representación común	→	fraude · crédito · recomendación
STRIPE · 2025	Payments Foundation Model · ataques detectados: 59 % → 97 %.				
NUBANK · 2026	nuFormer · en producción para decisiones de crédito.				
PLAID · 2026	Transaction Foundation Model · +48 % en clasificación de ingresos.				
REVOLUT · 2026	PRAGMA · 2,3× en identificación del riesgo de impago.				

Arquitectura emergente: modelos generales para lenguaje y agentes + modelos propios para comprender el comportamiento financiero.

FUENTES [Uber \(2026\)](#); [Block / Goose](#); [Coinbase y DoorDash / Fortune \(2026\)](#); [Perplexity \(2025\)](#). [Stripe \(2025\)](#); [Nubank \(2026\)](#); [Plaid \(2026\)](#); [Revolut / NVIDIA \(2026\)](#).

02 · SELECCIÓN Y MEDICIÓN

Taxonomía operativa de modelos.

Un mismo modelo puede combinar varias de estas dimensiones; las categorías no son excluyentes.

ARQUITECTURA	VARIANTES denso · mezcla de expertos (MoE)	EFFECTO OPERATIVO parámetros activos · memoria total · compatibilidad
ADAPTACIÓN	VARIANTES base · instrucciones/razonamiento · fine-tuning completo · LoRA/QLoRA · destilado	EFFECTO OPERATIVO comportamiento · dominio · tamaño del estudiante
MODALIDAD	VARIANTES texto · visión-lenguaje · audio · multimodal	EFFECTO OPERATIVO entradas · módulos adicionales · preprocesamiento

Estas dimensiones pueden existir en modelos propietarios o de pesos abiertos. El acceso determina cuáles puede inspeccionar y configurar el equipo.

SIGLAS

MoE: mezcla de expertos · LoRA: adaptación de bajo rango · QLoRA: adaptación de bajo rango cuantizada.

FUENTES [DeepSeek-AI \(2024\)](#); [Nie et al. \(2025\)](#).

Taxonomía operativa de modelos · II.

Representación, generación y residencia determinan cómo se ejecuta el modelo y qué recursos necesita.

REPRESENTACIÓN	VARIANTES 16 bits · 8 bits · 4 bits · precisión mixta	EFFECTO OPERATIVO disco · RAM/VRAM · ancho de banda · calidad
GENERACIÓN	VARIANTES autorregresiva · decodificación especulativa · predicción multitoken (MTP) · difusión	EFFECTO OPERATIVO tokens aceptados por paso · latencia de decodificación
RESIDENCIA DE PESOS	VARIANTES GPU · GPU + RAM · GPU + RAM + SSD NVMe (p. ej., Colibrì) NVMe (Non-Volatile Memory Express) es el protocolo de acceso al SSD; el runtime transfiere los pesos desde el SSD hacia RAM o GPU cuando los necesita.	EFFECTO OPERATIVO modelos mayores · más movimiento de datos · mayor latencia

La residencia amplía la capacidad utilizable, pero desplaza datos entre memorias con velocidades diferentes.

SIGLAS

MTP: predicción de múltiples tokens · GPU: unidad de procesamiento gráfico · RAM: memoria principal · VRAM: memoria de la GPU · SSD: unidad de estado sólido.

FUENTES [Nie et al. \(2025\)](#); [Sheng et al. \(2023\)](#); [JustVugg \(2026\)](#).

Modelos densos y modelos de mezcla de expertos.

La diferencia operativa está en cuántos parámetros participan en el procesamiento de cada token.

MODELO DENSO

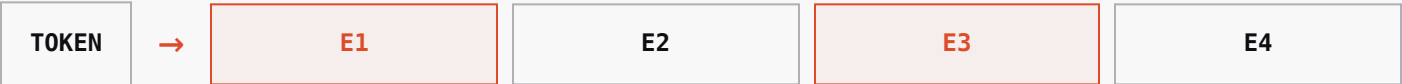
Cada token atraviesa las mismas capas y activa los parámetros correspondientes de cada bloque.



PARÁMETROS TOTALES	OPERACIÓN
aproximadamente iguales a los activos	ruta uniforme y compatibilidad más simple

MEZCLA DE EXPERTOS · MoE

Un enrutador selecciona un subconjunto de expertos para cada token.



PARÁMETROS TOTALES	OPERACIÓN
mayores que los parámetros activos	routing, comunicación y paralelismo de expertos

MoE reduce el cómputo activo por token; no reduce en la misma proporción la memoria necesaria para mantener disponibles todos los expertos.

FUENTE [DeepSeek-AI \(2024\), arquitectura MoE.](#)

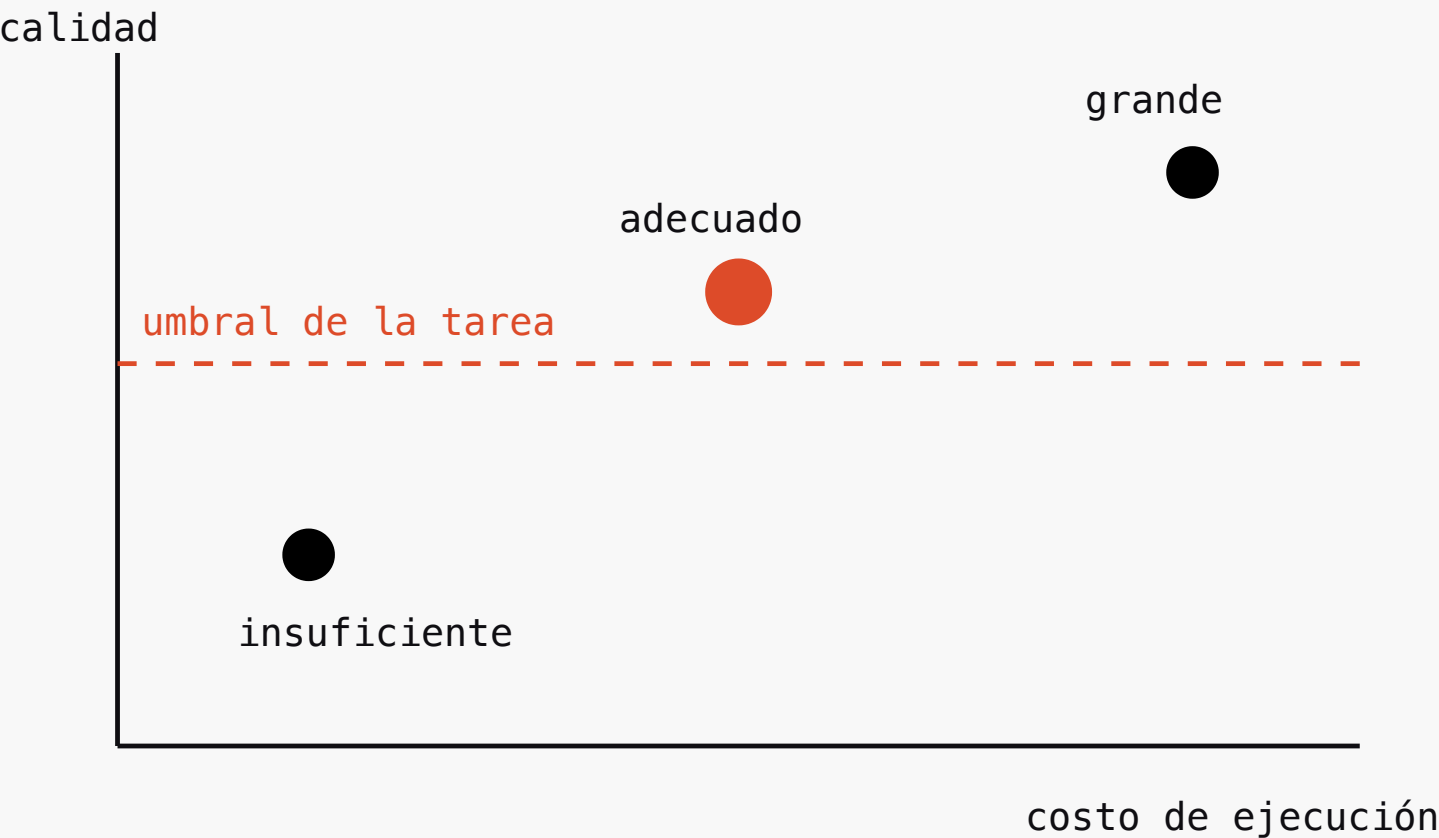
02 · SELECCIÓN Y MEDICIÓN

Selección del modelo.

A igualdad de arquitectura, hardware y optimizaciones, un modelo menor suele ser más rápido y económico.

La decisión de mayor impacto es elegir el modelo más pequeño y sencillo de ejecutar que alcance el umbral de calidad de la tarea.

La arquitectura elegida también determina la profundidad del soporte disponible en motores y herramientas de rendimiento.



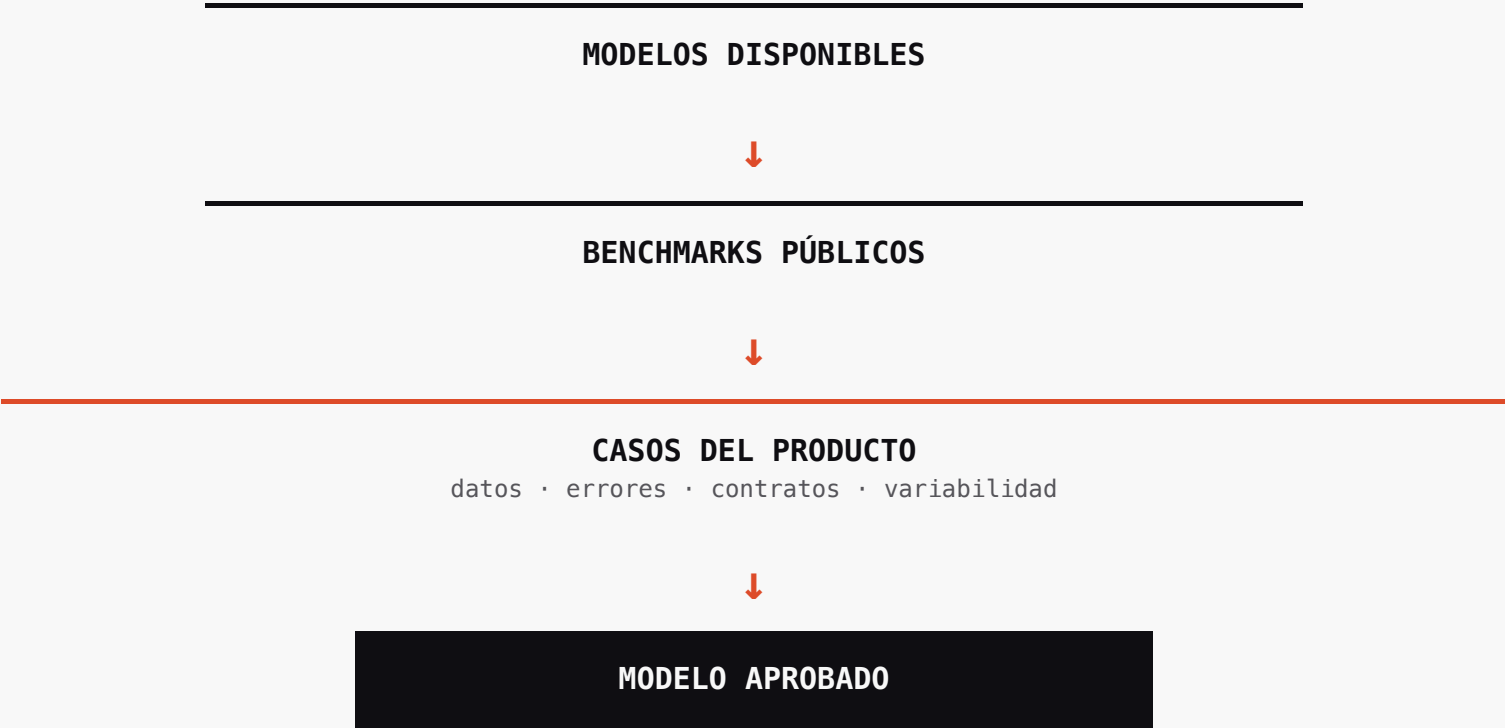
02 · SELECCIÓN Y MEDICIÓN

Evaluación del modelo.

Una evaluación de modelo, o eval, mide sistemáticamente si un modelo resuelve la tarea.

Los benchmarks públicos ayudan a formar una lista de candidatos. La evaluación del producto utiliza datos, errores y criterios propios para tomar la decisión.

Antes de optimizar, la evaluación cumple dos funciones: evita acelerar un modelo que no sirve y establece la línea base para detectar cambios posteriores de calidad.



Fine-tuning para calidad de dominio.

El fine-tuning adapta un modelo preentrenado a un caso de uso mediante nuevos datos.

Modifica valores de los pesos sin cambiar necesariamente la arquitectura general. En un dominio acotado y con evaluaciones sólidas, un modelo pequeño ajustado puede igualar a uno mayor en esa tarea.

La ventaja debe demostrarse con ejemplos representativos; no se deduce del proceso de ajuste.



02 · SELECCIÓN Y MEDICIÓN

Latencia y rendimiento.

LATENCIA

Tiempo que espera una solicitud para obtener una respuesta.

RENDIMIENTO

Cantidad total de trabajo que el sistema procesa por unidad de tiempo.

Reducir la espera individual y aumentar el volumen total son objetivos diferentes. La prioridad depende de si la carga es interactiva, por lotes o una combinación.

LATENCIA

UNA SOLICITUD

RESPUESTA

tiempo de espera individual

RENDIMIENTO

trabajo total por unidad de tiempo

02 · SELECCIÓN Y MEDICIÓN

Tiempo hasta el primer token y tokens por segundo.

Tiempo hasta el primer token, TTFT por *time to first token*.
Cuánto espera una persona hasta recibir el primer token. Re-
fleja principalmente la fase de prefill.

TOKENS POR SEGUNDO, TPS

Cuántos tokens recibe después del primero. Como métrica por
usuario describe latencia; como total del servicio describe
rendimiento.

Latencia entre tokens, ITL por *inter-token latency*. Tiempo
entre dos tokens consecutivos.

En llamadas a herramientas importa el tiempo total de
respuesta.



Si el TTFT fuera cero, una empresa podría activar una GPU durante un minuto para resolver una tarea y liberarla inmediatamente después. Como el arranque desde cero exige aprovisionar la GPU y cargar el modelo, los servicios interactivos suelen mantener el modelo cargado para evitar que el usuario perciba una respuesta lenta.

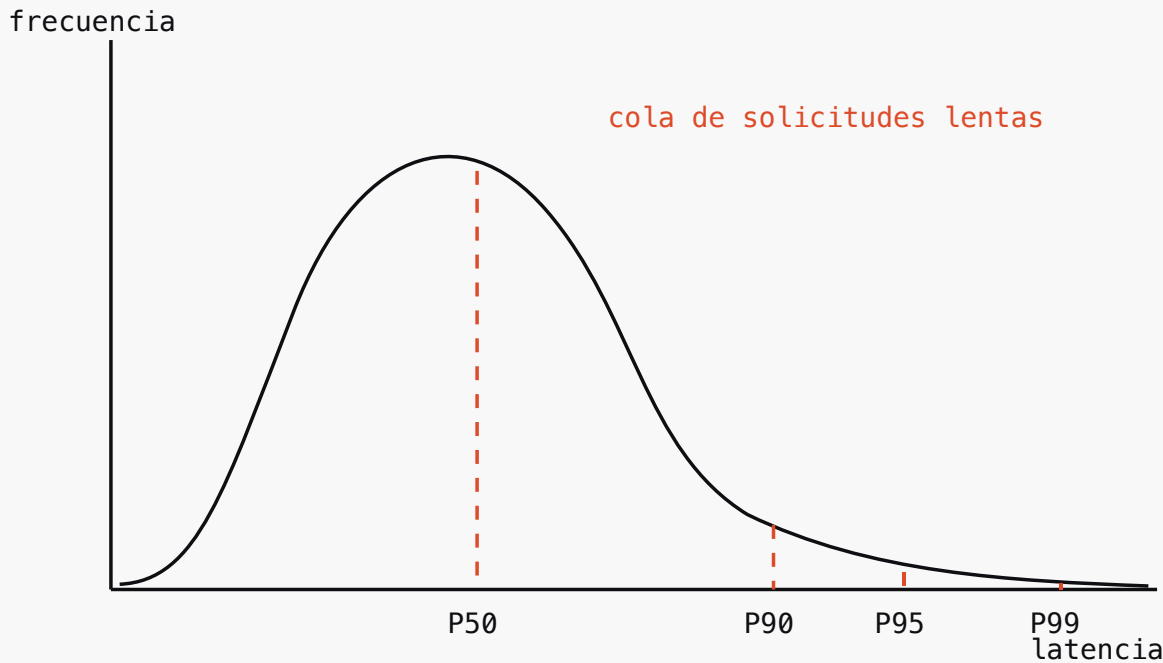
02 · SELECCIÓN Y MEDICIÓN

Percentiles de latencia.

La latencia suele seguir una distribución sesgada a la derecha: la mayoría de las solicitudes se concentra cerca del centro y unas pocas tardan mucho más.

P50 es la mediana: una de cada dos solicitudes es más lenta. P90: una de cada diez. P95: una de cada veinte. P99: una de cada cien.

El promedio por sí solo no describe la cola.



Suponga que llegan al mismo tiempo tres preguntas: «¿Cuál es la capital de Francia?», «¿Por qué el cielo es azul?» y «Resuelve el problema del milenio de las ecuaciones de Navier–Stokes». Si se procesan como un lote fijo, las tres respuestas se entregan cuando termina la tarea más lenta. En tráfico real, estos casos alargan la cola de latencia y se reflejan en percentiles como P95 y P99.

02 · SELECCIÓN Y MEDICIÓN

Métricas extremo a extremo.

El tiempo de inferencia mide el trabajo del modelo sobre el hardware.

La experiencia completa también incluye red, espera en cola, tokenización, validación, herramientas y recuperación.

Ambas mediciones son necesarias. Si la inferencia es rápida y la respuesta completa es lenta, el cuello de botella está fuera del modelo y debe buscarse en el sistema.



02 · SELECCIÓN Y MEDICIÓN

Líneas base de evaluación.

CALIDAD DEL MODELO

Compara una versión modificada con su antecedente para medir capacidad, formato, uso de herramientas y repeticiones.

CALIDAD DEL SISTEMA

Compara rutas de ejecución para medir validez, política, exactitud, recuperación y traza.

Una técnica del modelo y un cambio de harness son intervenciones distintas y deben evaluarse por separado.

EJE DEL MODELO	EJE DEL SISTEMA
mayor precisión→cuantizado	ruta directa→ruta con harness
calidad · herramientas · repetición	entregable · política · exactitud · traza

CUANTIZACIÓN ≠ INCORPORACIÓN DEL HARNESS

02 · SELECCIÓN Y MEDICIÓN

Evaluación de flujos agénticos.

Una evaluación agéntica observa más que la respuesta final:

- terminación correcta;
- formato y contrato de salida;
- selección y uso de herramientas;
- cumplimiento de políticas;
- repeticiones o bucles;
- recuperación después de un fallo; y
- consistencia entre ejecuciones.

Los casos difíciles y los fallos reales del producto deben convertirse en pruebas repetibles antes de optimizar el modelo o el sistema.



FUENTE [Hashimoto \(2026\)](#).

03

Mecánica del modelo.

¿Qué ocurre durante una inferencia?

03 · MECÁNICA DEL MODELO

Mecánica de inferencia de un LLM.

En los modelos de lenguaje autorregresivos, que dominan los sistemas actuales, la generación avanza token por token.

Cada token nuevo se produce a partir de todos los tokens anteriores. La entrada se procesa primero y la generación continúa un token por paso hasta producir un token de terminación o alcanzar un límite configurado.



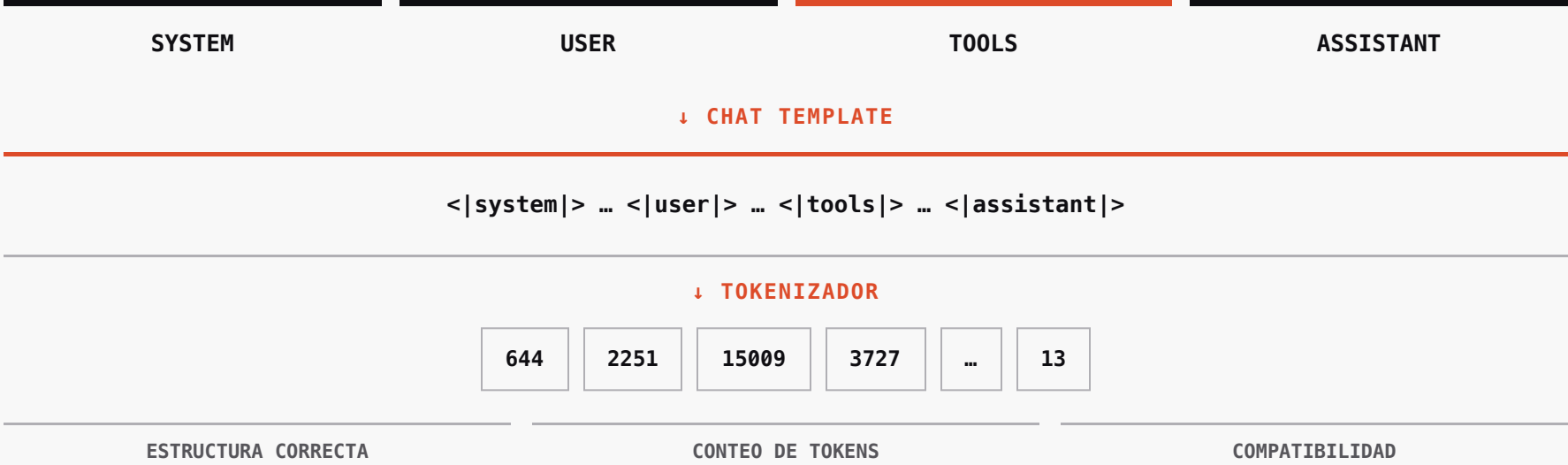
EL TOKEN GENERADO SE INCORPORA AL CONTEXTO DEL SIGUIENTE PASO

Chat template y tokenización.

Una aplicación organiza la interacción como mensajes, roles y herramientas; el modelo recibe una única secuencia de tokens.

El **chat template** serializa esa estructura con los tokens de control que el modelo aprendió durante el ajuste. Después, el tokenizador convierte la secuencia en identificadores del vocabulario.

Una plantilla incorrecta puede alterar roles, herramientas o terminación sin producir un error explícito. La cantidad de tokens resultante afecta contexto, costo y latencia.



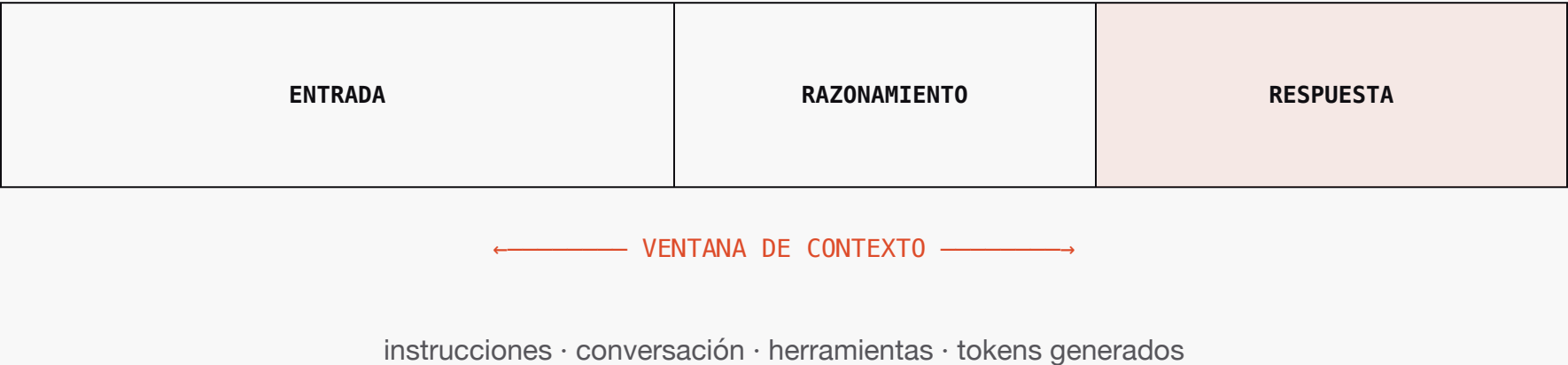
FUENTE [Hugging Face \(2026\), Chat templates.](#)

Secuencias y ventana de contexto.

Una solicitud puede contener tres secuencias:

Entrada: instrucciones, conversación, contexto y herramientas. Razonamiento: salida intermedia opcional. Respuesta: tokens entregados por el modelo.

Las tres comparten la ventana de contexto, que limita el total procesado y generado. La solicitud también puede fijar un máximo específico para la respuesta.



Arquitectura de un LLM y config.json.

La arquitectura especifica las operaciones y dimensiones que el **runtime de inferencia** debe reconstruir.

En Hugging Face, `config.json` declara la clase del modelo, profundidad, ancho, atención, contexto y uso de caché. Es el contrato estructural que conecta los pesos con una implementación compatible.

Una arquitectura puede admitir varios tamaños, variantes instruct y fine-tunes. Compartir operaciones permite reutilizar soporte del runtime y kernels; el tamaño, la precisión y la configuración siguen determinando memoria y rendimiento.

config.json

QWEN3.8-27B · CAMPOS SELECCIONADOS

architectures	Qwen3_5ForConditionalGeneration	clase que debe implementar el runtime
num_hidden_layers	64	profundidad de la red
hidden_size	5120	ancho del estado oculto
attention_heads / kv_heads	24 / 4	estructura de la atención
max_position_embeddings	262 144	límite arquitectónico de posiciones
use_cache	true	habilita el uso de caché KV

TOKENS

→

EMBEDDINGS

→

BLOQUE TRANSFORMER × 64

→

LM HEAD → LOGITS

ATENCIÓN + RESIDUAL

MLP + RESIDUAL

03 · MECÁNICA DEL MODELO

Atención.

Cada posición se transforma en tres vectores. La **consulta, Q**, expresa qué información necesita la posición actual. Las **claves, K**, permiten determinar qué posiciones son relevantes. Los **valores, V**, contienen la información que se combina de acuerdo con esa relevancia.

La similitud entre Q y K produce pesos de atención; esos pesos determinan cuánto aporta cada V.

En autoatención causal, una posición solo puede atender a sí misma y a posiciones anteriores.

- Q

qué información busca la posición actual
- K

qué posiciones anteriores son relevantes
- V

qué información aporta cada posición



03 · MECÁNICA DEL MODELO

Caché de claves y valores.

La caché KV, por *key-value cache*, conserva las claves y valores calculados para los tokens anteriores.

Se construye durante prefill y se consulta y actualiza durante decode. Sin ella, cada token nuevo obligaría a recalcular toda la secuencia previa.

La caché acelera la generación, pero ocupa memoria de GPU y crece con el contexto y la concurrencia.

PASO 1	PASO 2	PASO n
$K_1 \cdot V_1$	K_1, V_1 reutilizados $+ K_2 \cdot V_2$	$K_1 \dots K_{n-1}$ reutilizados $+ K_n \cdot V_n$
MEMORIA DE CACHÉ ↑ CON CONTEXTO × CONCURRENCIA		

03 · MECÁNICA DEL MODELO

Prefill y decode.

Prefill procesa conjuntamente todos los tokens de entrada, construye la caché KV del prompt y produce los logits con los que se selecciona el primer token de salida. Suele estar limitado por la **capacidad de cómputo de la GPU o del acelerador** y determina gran parte del **tiempo hasta el primer token (TTFT)**.

Decode genera la respuesta de forma autorregresiva: produce un token nuevo por iteración, reutiliza la caché KV y la actualiza. Con lotes pequeños o medianos, suele estar limitado por el **ancho de banda de la memoria de la GPU (VRAM)** y determina la **latencia entre tokens (ITL)**.



03 · MECÁNICA DEL MODELO

Cómo se selecciona el siguiente token.

1. **Logits.** La capa de salida asigna a cada token un puntaje sin normalizar. Un valor mayor indica una preferencia relativa más alta; todavía no es una probabilidad.
2. **Temperatura.** Reescala los logits antes de softmax. Una temperatura baja amplía sus diferencias y favorece los tokens dominantes; una temperatura alta reduce esas diferencias y reparte la probabilidad entre más tokens.
3. **Softmax.** Convierte los logits reescalados en probabilidades que suman 100 %.
4. **Top-k o top-p.** Top-k conserva los **k** tokens más probables; top-p conserva el conjunto mínimo cuya probabilidad acumulada alcanza **p**.
5. **Selección del token.** Se toma una muestra entre los candidatos restantes o, en modo determinista, se elige el token con mayor probabilidad.
6. **Siguiente paso.** El token elegido se añade al contexto y comienza el siguiente paso de **decode**.

EJEMPLO ILUSTRATIVO

contexto: «El usuario quiere consultar su»

TOKEN	LOGIT	PROBABILIDAD	TOP-P = 0,90
saldo	3,8	56 %	●
cuenta	3,0	25 %	●
movimientos	2,3	12 %	●
crédito	1,8	7 %	×

TEMPERATURA
ajusta los logits

→

SOFTMAX
produce probabilidades

→

TOP-K / TOP-P
filtra candidatos

CANDIDATOS RESTANTES → MUESTREO O ARGMAX → «saldo»

04

Técnicas.

¿Qué cuello de botella resuelve cada familia de optimización?

04 · TÉCNICAS

Mapa de técnicas de inferencia.

TÉCNICA	QUÉ HACE	CUELLO PRINCIPAL
Procesamiento por lotes	intercala solicitudes token a token	rendimiento
Caché	reutiliza estados de prefijos compartidos	prefill repetido
Cuantización	reduce precisión selectivamente	memoria y cómputo
Especulación	propone y valida varios tokens	decode serial
Paralelismo	distribuye el modelo entre GPU	capacidad o latencia
Desagregación	separa prefill y decode	escalamiento independiente

04 · TÉCNICAS

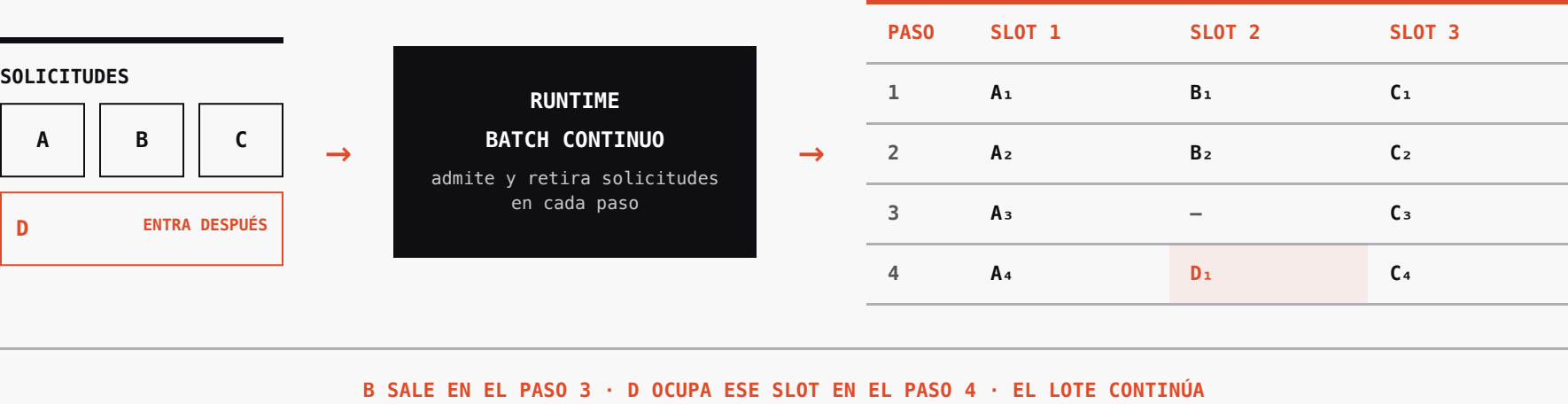
Procesamiento por lotes [Batching].

El procesamiento por lotes ejecuta solicitudes entrantes en paralelo e intercala su avance token a token.

Un lote mayor utiliza mejor la GPU y aumenta el rendimiento total. También puede aumentar la espera individual y consumir el cálculo que otras técnicas necesitan.

El runtime debe ajustar el lote a la concurrencia, las longitudes y el objetivo de latencia.

Hacer inferencia de uno en uno es como usar un autobús de 50 pasajeros para transportar a una sola persona: el viaje es rápido, pero el costo por persona es altísimo y el vehículo está infrautilizado. El batching llena los asientos del autobús.



Decodificación especulativa.

Un generador de borrador propone varios tokens. El modelo objetivo los verifica en paralelo, acepta el prefijo correcto y genera un reemplazo después del primer rechazo.

La técnica puede producir más de un token por paso de decode sin cambiar la distribución final esperada.

El beneficio depende del costo del borrador, su longitud y la tasa de aceptación.



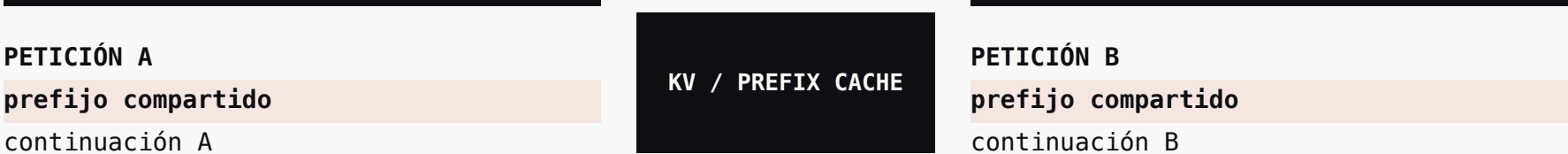
04 · TÉCNICAS

Caché y reutilización de prefijos.

La caché KV evita recalcular tokens previos dentro de una solicitud.

La caché de prefijos reutiliza ese trabajo entre solicitudes que comparten exactamente el mismo inicio. Como cada token influye en los siguientes, una diferencia temprana elimina la coincidencia posterior.

El beneficio depende del orden del contexto, la memoria disponible y el enrutamiento hacia una réplica que conserve la caché.



menos prefill ↔ más memoria y afinidad de enrutamiento

04 · TÉCNICAS

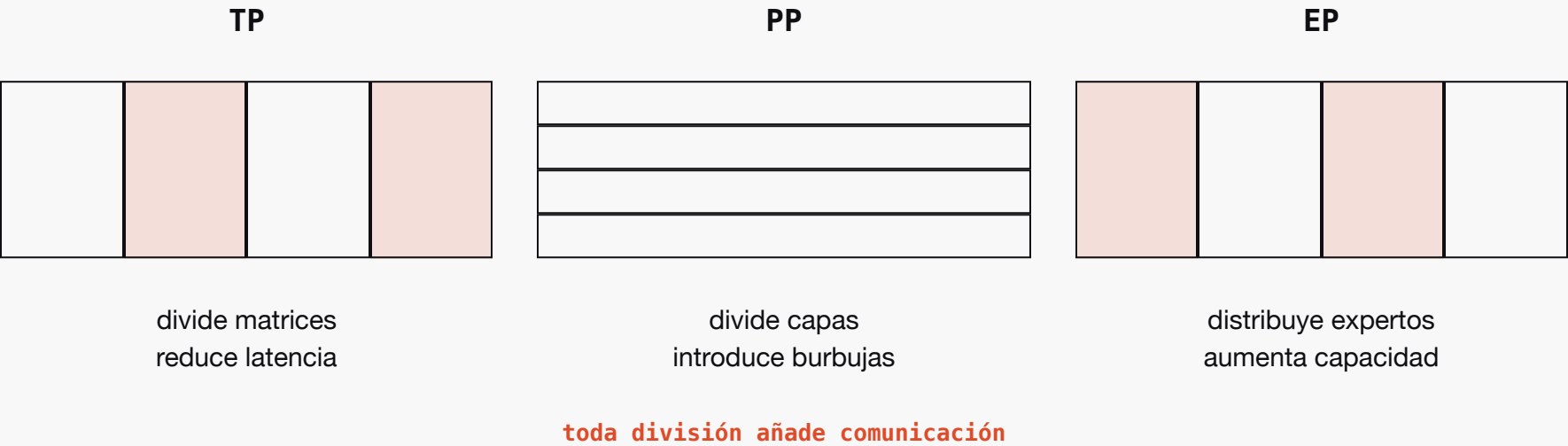
Paralelismo del modelo.

Paralelismo por etapas, PP por *Pipeline Parallelism*, divide las capas entre GPU y puede introducir esperas entre etapas.

Paralelismo tensorial, TP por *Tensor Parallelism*, divide las matrices de cada capa. Suele reducir latencia dentro de un nodo, pero requiere sincronización frecuente.

Paralelismo de expertos, EP por *Expert Parallelism*, distribuye expertos completos de modelos MoE. Favorece rendimiento y escala mejor entre nodos.

Toda forma de paralelismo intercambia capacidad por comunicación; la topología de los enlaces determina el resultado.



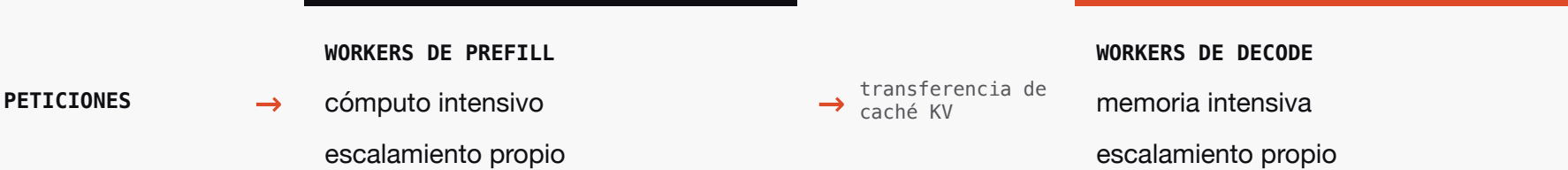
04 · TÉCNICAS

Desagregación.

La desagregación separa prefill y decode en motores y trabajadores diferentes.

El trabajador de prefill procesa la entrada, produce el primer token y transfiere la caché KV. El trabajador de decode genera los tokens restantes.

La técnica permite especializar y escalar cada fase, pero exige varias GPU, transferencia eficiente y tráfico elevado. Baseten la recomienda para modelos grandes, entradas largas y cargas dominadas por prefill.



05

Cuantización aplicada.

¿Qué se gana, qué se arriesga y qué hizo Alchemist?

05 · CUANTIZACIÓN APLICADA

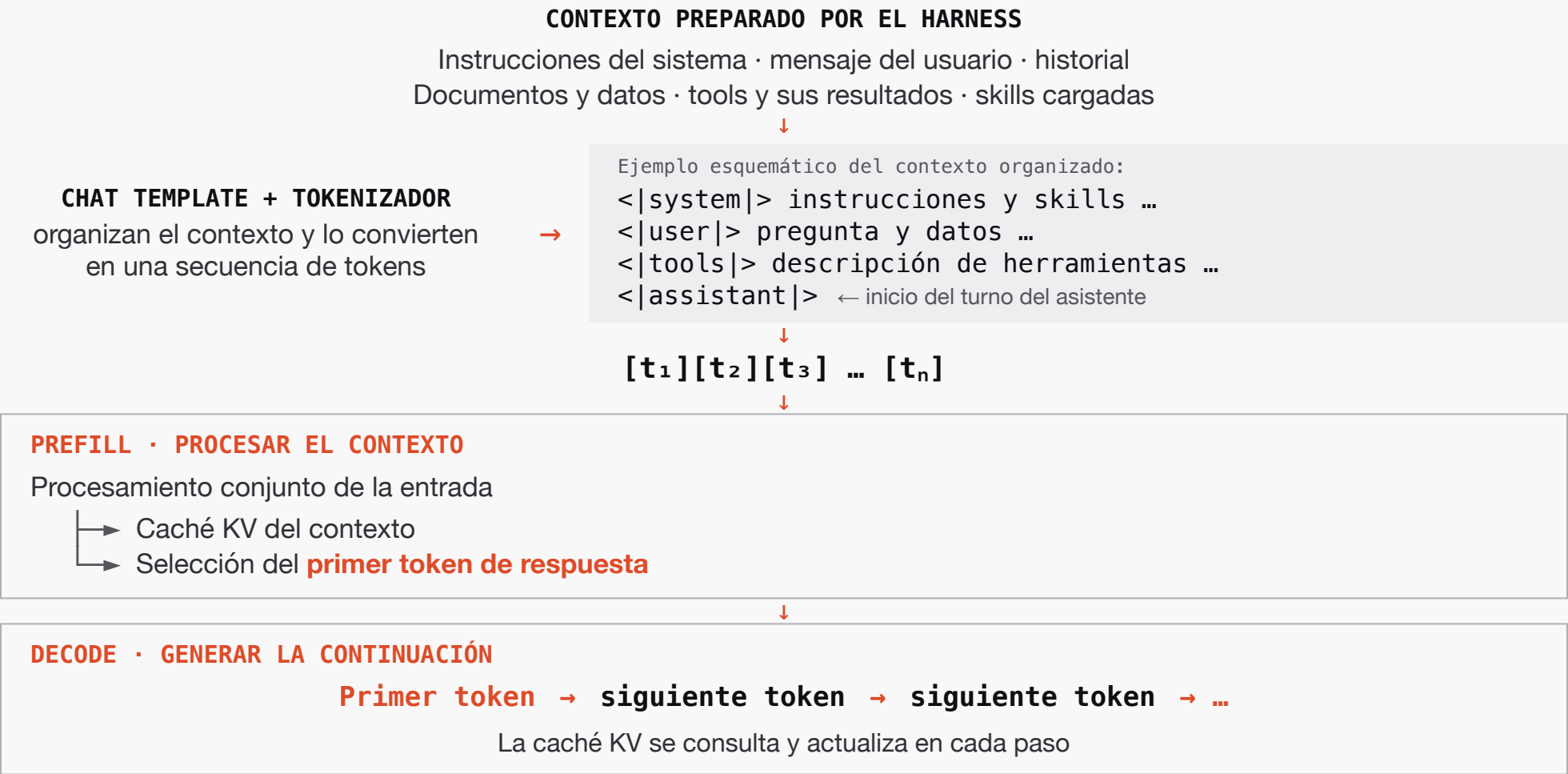
Prefill y decode.

PREFILL · PROCESAR EL CONTEXTO

El modelo procesa conjuntamente los tokens de entrada, construye la caché de claves y valores —KV— y produce los puntajes con los que se selecciona el primer token de respuesta.

DECODE · GENERAR LA CONTINUACIÓN

El modelo genera los siguientes tokens uno por uno. Cada paso utiliza el token anterior y consulta y actualiza la caché KV hasta completar la respuesta.



05 · CUANTIZACIÓN APLICADA

Cuantización.

La cuantización representa pesos, activaciones u otros valores con menor precisión que su formato original.

En prefill puede aprovechar unidades de cálculo de menor precisión. En decode reduce los datos que deben leerse desde memoria.

También reduce almacenamiento y libera espacio para caché u otras optimizaciones. A cambio, introduce errores de aproximación que pueden acumularse y modificar la calidad de la salida.



05 · CUANTIZACIÓN APLICADA

Punto flotante.

Queremos guardar -6,5 en FP16.

En FP16, la base siempre es 2. Lo que cambia es el exponente.

signo

1 implícito

exponente

fracción: 0,625

base: 2

-

1

,

625

×

2

²

COMPONENTE	BITS	CÓDIGO BINARIO ALMACENADO	CÓMO SE INTERPRETA
Signo	1	1	Negativo. El signo se guarda por separado.
Exponente	5	10001	El código vale 17. Restamos 15 y recuperamos 2. Dos elevado a dos = 4: ese es el multiplicador.
Fracción	10	1010000000	1,625 = 1 + 0,625. La fracción almacena la parte 0,625. El 1 inicial es implícito: no se almacena en los números normalizados.

Resultado: -1,625 × 4 = -6,5

Total: 1 + 5 + 10 = 16 bits

POSICIÓN TRAS LA COMA BINARIA	PRIMERA	SEGUNDA	TERCERA
Cuánto aporta si tiene un 1	0,5	0,25	0,125
Bit del ejemplo	1	0	1
Aporte al valor	0,5	0	0,125

Sumamos los aportes: 0,5 + 0 + 0,125 = 0,625.

FP16 reserva diez bits para la fracción: 101 y siete ceros. Los ceros completan el campo sin cambiar el valor.

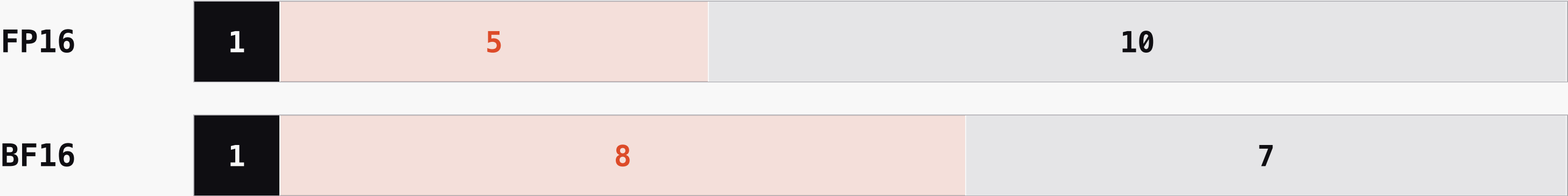
Esos tres bits representan solo la fracción. Con signo y exponente, -6,5 ocupa 16 bits.

05 · CUANTIZACIÓN APLICADA

Formatos numéricos.

FP16 y BF16 ocupan el mismo espacio por valor, pero distribuyen sus bits de forma diferente.

■ signo ■ exponente ■ fracción



FP16 · Floating Point de 16 bits

10 bits de fracción: pasos más pequeños entre valores a una misma escala.

BF16 · Brain Floating Point

8 bits de exponente: un rango de magnitudes más amplio.

AMBOS OCUPAN 16 BITS POR VALOR

05 · CUANTIZACIÓN APLICADA

Punto flotante y enteros cuantizados.

FP · FLOATING POINT

FP16 · FP8 · FP4

Los bits se interpretan como **signo, exponente y fracción**.

EJEMPLO EN FP16

$-1,625 \times 2^2 = -6,5$

El signo, la fracción y el exponente permiten recuperar el valor representado.

La separación entre valores representables cambia con la magnitud. No es uniforme en todo el rango.

FP4 e INT4 utilizan cuatro bits, pero interpretan esos bits de manera diferente.

Los formatos flotantes de baja precisión también pueden utilizar escalas adicionales.

INT · INTEGER

INT8 · INT4

Se guarda un **código entero**. Una escala permite reconstruir una aproximación del valor.

CÓDIGO × ESCALA = VALOR RECONSTRUIDO

$3 \times 0,1 = 0,3$

INT4 guarda el código 3, no directamente 0,3.

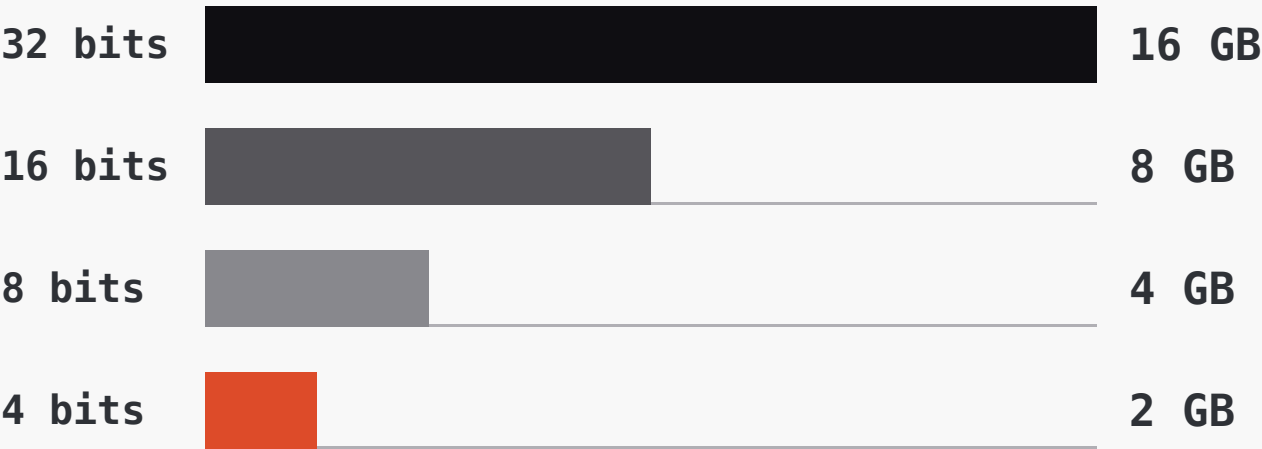
La escala se almacena aparte y puede compartirse entre varios valores. Para una escala fija, los pasos son iguales. Según el esquema, también se aplica un punto cero.

05 · CUANTIZACIÓN APLICADA

Memoria teórica y memoria real.

Los mismos 4.000 millones de pesos

Menos bits por peso reducen el espacio ocupado.



Número de pesos × bits por peso ÷ 8 = bytes

GB decimales: 1 GB = 1.000 millones de bytes.
Estas cifras solo cuentan los valores de los pesos.

Para ejecutar el modelo se necesita más memoria

- Parámetros de cuantización.** Escalas y, según el método, desplazamientos que permiten reconstruir los pesos aproximados.
- Activaciones.** Resultados intermedios que producen las capas al procesar la entrada.
- Caché KV y otros estados.** Información de los tokens ya procesados que el modelo conserva y reutiliza.
- Memoria temporal de ejecución.** Espacio de trabajo que el programa usa mientras realiza los cálculos.

La memoria total depende también de la longitud del contexto, las solicitudes simultáneas y el programa de ejecución.

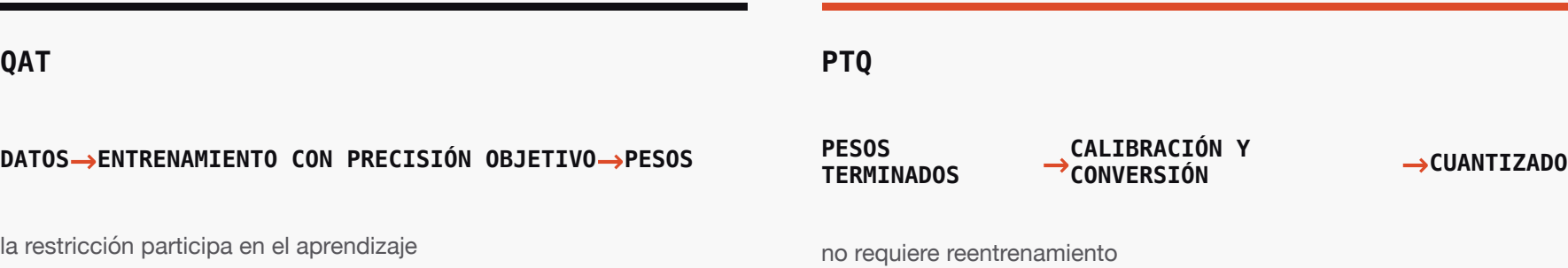
05 · CUANTIZACIÓN APLICADA

Cuantización durante y después del entrenamiento.

Entrenamiento consciente de cuantización, QAT por *Quantization-Aware Training*. Incorpora o simula la precisión objetivo durante el entrenamiento para que los pesos aprendan bajo esa restricción.

Cuantización posterior al entrenamiento, PTQ por *Post-Training Quantization*. Convierte pesos ya entrenados y calcula factores de escala mediante calibración.

PTQ evita reentrenar, pero la calibración y la evaluación siguen siendo necesarias para preservar calidad.



05 · CUANTIZACIÓN APLICADA

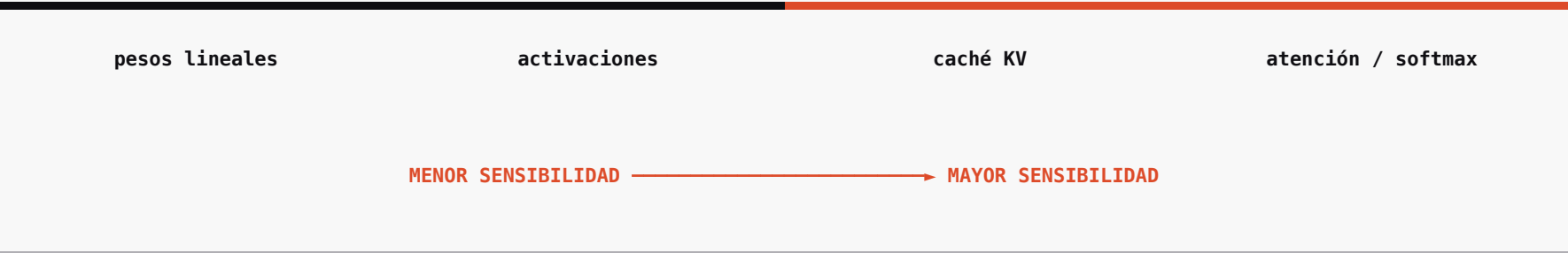
Sensibilidad por componente.

Los componentes toleran la reducción de precisión de manera diferente.

Las matrices lineales suelen ser las menos sensibles. Las activaciones requieren calibración. La caché KV acumula error entre tokens. Las operaciones de atención, especialmente softmax, son las más sensibles y suelen conservar mayor precisión.

Las capas de entrada y salida también pueden requerir protección.

Esta jerarquía orienta; la evaluación decide.



entrada · salida · normalizaciones pueden requerir mayor precisión

05 · CUANTIZACIÓN APLICADA

Precisión mixta.

Precisión mixta. Usar distintas representaciones numéricas en diferentes partes de un mismo modelo.

En Alchemist, las matrices de proyección combinan las características internas y guardan sus pesos como **códigos enteros de 4 bits**.

La tabla de vocabulario usa **códigos enteros de 6 bits** y se comparte entre la entrada y la salida.

Las normalizaciones, otros parámetros no cuantizados y el módulo de visión conservan **BF16: punto flotante de 16 bits**.

DISTRIBUCIÓN DE LOS PESOS EN ALCHEMIST



Los 4 y 6 bits describen cómo se guardan los pesos.
Las operaciones no tienen que usar esa misma precisión.

Los códigos enteros de 4 y 6 bits se combinan con escalas y desplazamientos. No son formatos FP4 ni FP6.

FUENTE [Datastrat \(2026\). Alchemist.](#)

05 · CUANTIZACIÓN APLICADA

Evaluación del impacto en calidad.

Una cuantización se compara con el modelo de precisión original mediante perplejidad, benchmarks públicos y evaluaciones específicas del producto.

PERPLEJIDAD

Mide cuánta incertidumbre asigna el modelo al siguiente token en un conjunto de texto. Un valor menor indica una mejor predicción de ese conjunto; solo es comparable usando los mismos datos, tokenizador y procedimiento.

En flujos agénticos también se observan terminación, formato, uso de herramientas, repeticiones y cumplimiento de contratos.

El estándar de producción es que la pérdida permanezca dentro del umbral aceptado y del ruido de evaluación. Una mejora promedio no compensa el deterioro de una tarea crítica.



05 · CUANTIZACIÓN APLICADA

Cuantización extrema y formatos de distribución.

Una representación binaria conserva dos estados por peso. Una ternaria añade cero y requiere teóricamente $\log_2(3) = 1,585$ bits. El archivo real también almacena escalas, empaquetado y metadatos.

GGUF es un formato habitual para distribuir modelos cuantizados en ejecución local. Formato, método y runtime son decisiones distintas: dos archivos etiquetados con los mismos bits pueden utilizar diferente granularidad, metadatos, operadores y producir distinto desempeño.

BINARIO		TERNARIO	
−1 · +1		−1 · 0 · +1	
2 estados		log ₂ (3) = 1,585 bits teóricos	
MÉTODO	qué se cuantiza y cómo se calibra		
FORMATO · GGUF	cómo se empaqueta y distribuye		
RUNTIME	qué operaciones ejecuta con eficiencia		

FUENTES [PrismML \(2026\)](#); [Unsloth \(2026\)](#).

Caso Alchemist: arquitectura y repositorio.

Alchemist parte de un modelo de cuatro mil millones de pa-
rámetros con ajuste agéntico previo.

Su config.json declara 32 capas y una arquitectura híbrida
que intercala tres capas de atención lineal con una de aten-
ción completa: 24 lineales y 8 completas.

El repositorio contiene configuración, tokenizador, plantilla,
licencia, scripts, índice y archivos safetensors. El ejemplo
utiliza el módulo de lenguaje; el módulo de visión permanece
en BF16.

Este es el caso aplicado, no la definición general.



FUENTES Galvis (2026); Datastrat (2026).

05 · CUANTIZACIÓN APLICADA

Caso Alchemist: método de cuantización.

- 01

Reescalar 216 matrices. Aumentar los pesos asociados con determinadas características de entrada y dividir esas entradas por el mismo factor, antes de redondear.
- 02

Integrar la compensación. Incorporar esa división en los parámetros de una capa vecina, sin añadir un cálculo durante la inferencia.
- 03

Cuantizar por grupos de 128 pesos. Las 248 proyecciones usan códigos enteros de 4 bits. De ellas, 32 se cuantizan sin reescalar. La tabla de vocabulario compartida usa 6 bits.
- 04

Evaluar y conservar el rango. Recortar los valores extremos aumentó las repeticiones de llamadas fallidas a herramientas. Por eso se descartó.

COMPENSAR CONSERVA EL CÁLCULO

ANTES

$2 \times 0,5 = 1$

entrada x peso

$ENTRADA \div 4 \cdot PESO \times 4$

$0,5 \times 2 = 1$

mismo resultado antes de redondear

GRUPO DE CUANTIZACIÓN

En Alchemist, un grupo reúne 128 pesos que comparten una escala y un desplazamiento.

128 códigos enteros

+ escala en BF16

+ desplazamiento en BF16

Escala: distancia entre dos valores consecutivos.

Desplazamiento: valor correspondiente al código cero.

Peso aproximado = código x escala + desplazamiento

4 bits por código

$(128 \times 4 + 16 + 16) \div 128$

4,25 bits por peso

6 bits por código

$(128 \times 6 + 16 + 16) \div 128$

6,25 bits por peso

FUENTES Galvis (2026); Datastrat (2026); MLX Contributors (s. f.).

05 · CUANTIZACIÓN APLICADA

Caso Alchemist: resultado y criterio de aprobación.

MÓDULO DE LENGUAJE

4,555 bits efectivos por peso · 2,395 GB.

MÓDULO DE VISIÓN

BF16 · 0,667 GB.

La demostración utiliza el módulo de lenguaje.

“Modelo de 4 bits” y “modelo de 2,4 GB” son resúmenes operativos, no descripciones completas del repositorio.

La enseñanza del caso es metodológica: comprimir significa decidir qué comportamiento conservar y aprobarlo mediante evaluaciones de la tarea.

LENGUAJE

4,555 bits efectivos por peso

2,395 GB

VISIÓN

BF16

0,667 GB

EVALUACIÓN DE LA TAREA → APROBAR / RECHAZAR

“4 bits” y “2,4 GB” son resúmenes operativos

FUENTES Galvis (2026); Datastrat (2026).

06

Harness y composición.

¿Cómo se convierte capacidad en un sistema operable y modular?

06 · HARNESS MODULAR

Capacidad del modelo y confiabilidad del sistema.

CAPACIDAD DEL MODELO

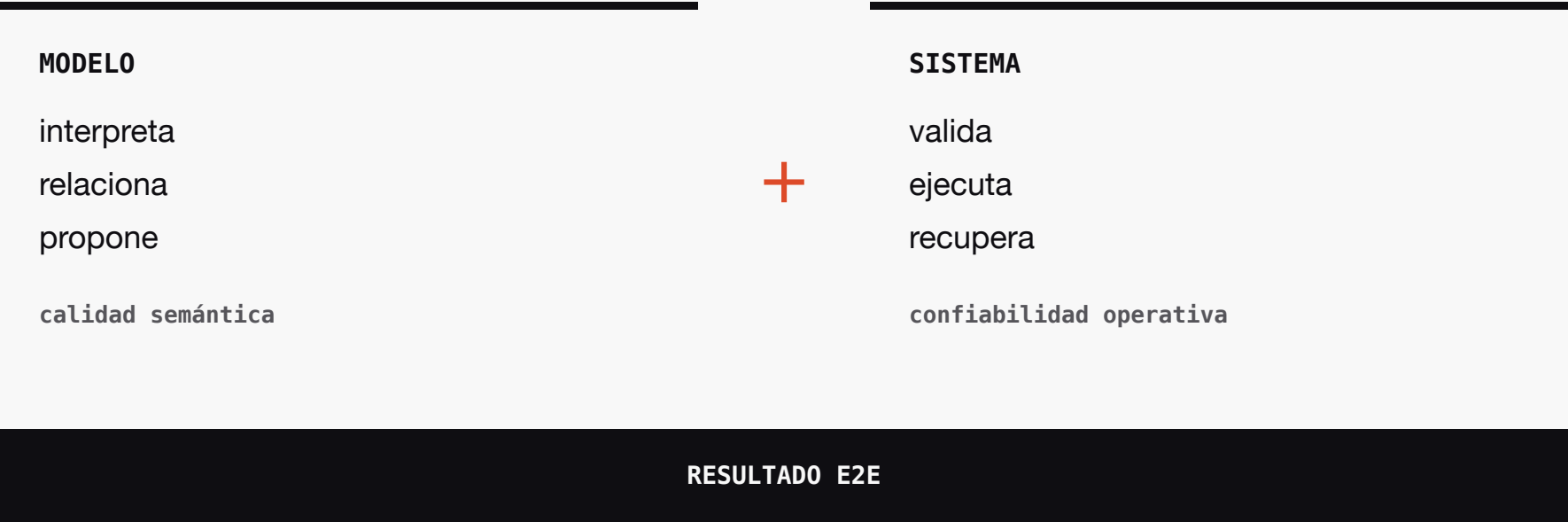
Calidad de la interpretación y de la propuesta.

CONFIABILIDAD DEL SISTEMA

Validez, terminación, cumplimiento de políticas, recuperación y trazabilidad.

Un modelo puede comprender correctamente la tarea y producir una salida truncada, inválida, repetitiva o imposible de ejecutar.

Las dos propiedades se relacionan, pero requieren preguntas, métricas y líneas base diferentes.



FUENTES [Hashimoto \(2026\)](#); [Trivedy \(2026\)](#).

06 · HARNESS MODULAR

Harness de un agente.

El harness es la arquitectura de ejecución y control que rodea al modelo.

Administra contexto y estado, proporciona herramientas, aplica permisos y políticas, verifica resultados, registra transiciones y decide cuándo continuar, reparar o terminar.

Puede operar con un modelo cerrado por API o con uno de pesos abiertos. Los pesos amplían el control sobre la ejecución; el harness aporta contratos, validación, recuperación y observabilidad.

HARNESS · ARQUITECTURA DE EJECUCIÓN Y CONTROL



FUENTES [Hashimoto \(2026\)](#); [Trivedy \(2026\)](#).

Caso aplicado: Alchemist + harness.

El caso utiliza una pregunta y transacciones sintéticas.

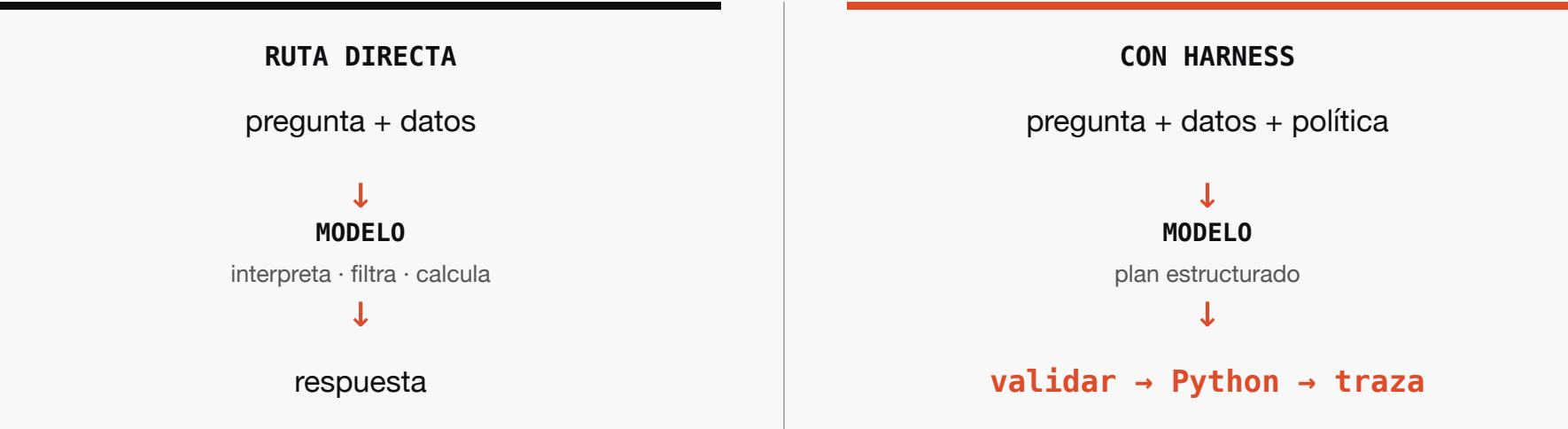
RUTA DIRECTA

Alchemist interpreta, selecciona y calcula sobre el registro.

RUTA CON HARNESS

El sistema aporta taxonomía y política; Alchemist propone un plan estructurado; un validador aplica reglas; Python selecciona y suma; la traza conserva el proceso.

Ambas rutas usan el mismo checkpoint y los mismos datos. La comparación muestra la externalización de política y cálculo; no atribuye al harness una mejora de capacidad del modelo.



FUENTES Galvis (2026); Datastrat, modelo Alchemist (2026).

Evaluación de la sesión 2.

ALCHEMIST + HARNESS MÍNIMO

01	02	03
¿La salida es entregable?	¿Cumple la política declarada?	¿Es correcta frente a una referencia?

CONTRATOS · VALIDACIÓN · RECUPERACIÓN · CÁLCULO · TRAZA

FUENTES Galvis (2026); Datastrat, modelo Alchemist (2026).

R

Referencias.

Referencias · A–G.

Aubakirova, M., & Midha, A. (2025, 4 de diciembre). *State of AI: An empirical 100 trillion token study with OpenRouter*. Andreessen Horowitz.
<https://a16z.com/state-of-ai/>

Datastrat. (2026a). *Agent A1 Alchemist 4-bit* [Modelo de lenguaje]. Hugging Face.
<https://huggingface.co/angelgalvis/agent-a1-alchemist-4bit>

Galvis, A. (2026). *Inference Masterclass I: Quantization/Harness-Es* [Notebook de Kaggle]. Kaggle.
<https://www.kaggle.com/code/angelgalvis/inference-masterclass-i-quantization-harness-es>

Belcak, P., Heinrich, G., Diao, S., Fu, Y., Dong, X., Muralidharan, S., Lin, Y. C., & Molchanov, P. (2025). *Small language models are the future of agentic AI* [Preprint]. arXiv.
<https://arxiv.org/abs/2506.02153>

DeepSeek-AI. (2025). *DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning* [Preprint]. arXiv.
<https://arxiv.org/abs/2501.12948>

Referencias · H–M.

Hashimoto, M. (2026, 5 de febrero). *My AI adoption journey*.
<https://mitchellh.com/writing/my-ai-adoption-journey>

Hugging Face. (s. f.-a). *Hugging Face Hub documentation*.
<https://huggingface.co/docs/hub/en/index>

Kiely, P. (2026). *Inference engineering*. Baseten Books.
<https://www.baseten.co/inference-engineering/>

Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the knowledge in a neural network* [Preprint]. arXiv.
<https://arxiv.org/abs/1503.02531>

Hugging Face. (s. f.-b). *Safetensors* [Repositorio de código]. GitHub.
<https://github.com/huggingface/safetensors>

Macatee, G. (2026, mayo). *AI infrastructure results in 2025 top expectations, forecast upgraded*. S&P Global Market Intelligence.
[Abrir fuente de S&P Global](#)

Referencias · O–U.

OpenRouter. (2026, 30 de junio). *DeepSeek V4 is earning agentic token share*.
<https://openrouter.ai/blog/insights/deepseek-v4-adoption/>

PrismML. (2026, 16 de abril). *Introducing Ternary Bonsai: Top intelligence at 1.58 bits*.
<https://prismml.com/news/ternary-bonsai>

Trivedy, V. (2026, 10 de marzo). *The anatomy of an agent harness*. LangChain.
<https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>

Open Source Initiative. (2024). *The Open Source AI Definition 1.0*.
<https://opensource.org/ai/open-source-ai-definition>

Shi, Y., Zhang, W., & Cui, T. (2026). *A programming paradigm for spatiotemporal composability* [Preprint]. arXiv.
<https://arxiv.org/abs/2608.25512>

Unsloth. (2026). *Unsloth Dynamic 2.0 GGUFs*.
<https://unsloth.ai/docs/basics/unsloth-dynamic-2.0-ggufs>

Referencias · Ecosistema I.

Block Open Source Program Office. (s. f.). *Accelerating open source AI at Block: Introducing the goose grant program*.

[Abrir fuente de Block](#)

Perplexity Team. (2025, 11 de febrero). *Meet the new Sonar: A blazing-fast model optimized for Perplexity Search*.

[Abrir fuente de Perplexity](#)

Gordon, N. (2026, 26 de julio). *China’s Moonshot, Z.AI, and DeepSeek are challenging U.S. AI labs —and beating them on cost*. Fortune.

[Abrir fuente de Fortune](#)

Ling, B., Huang, J., Liu, B., Cao, C., Vyas, A., & Zhang, P. (2026, 14 de abril). *Open source and in-house: How Uber optimizes LLM training*. Uber.

[Abrir fuente de Uber](#)

REFERENCIAS · APA 7

Referencias · Ecosistema II.

Nubank. (2026, 11 de junio). *Nubank details AI transformation strategy built on data, foundation models, and democratized financial advice.*
[Abrir fuente de Nubank](#)

Plaid. (2026, 2 de abril). *Building a transaction foundation model to power intelligent finance.*
<https://plaid.com/blog/building-transaction-foundation-model-intelligent-finance/>

NVIDIA. (2026). *Revolut builds a transaction foundation model with NVIDIA accelerated computing platform.*
<https://www.nvidia.com/en-eu/case-studies/revolut/>

Stripe. (2025, 15 de mayo). *Using AI to optimize payments performance with the Payments Intelligence Suite.*
[Abrir fuente de Stripe](#)

Referencias · Arquitecturas.

DeepSeek-AI. (2024). *DeepSeek-V3 technical report* [Preprint]. arXiv.
<https://arxiv.org/abs/2412.19437>

Hugging Face. (2026). *Chat templates*. Transformers documentation.
https://huggingface.co/docs/transformers/main/en/chat_templating

JustVugg. (2026). *colibri: Run models bigger than your machine* [Software]. GitHub.
<https://github.com/JustVugg/colibri>

Qwen. (2026). *Qwen3.8-27B: config.json* [Archivo de configuración de modelo]. Hugging Face.
<https://huggingface.co/Qwen/Qwen3.8-27B/blob/main/config.json>

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient finetuning of quantized LLMs* [Preprint]. arXiv.
<https://arxiv.org/abs/2305.14314>

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-rank adaptation of large language models* [Preprint]. arXiv.
<https://arxiv.org/abs/2106.09685>

Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., Zhou, J., Lin, Y., Wen, J.-R., & Li, C. (2025). *Large language diffusion models* [Preprint]. arXiv.
<https://arxiv.org/abs/2502.09992>

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Fu, D. Y., Xie, Z., Chen, B., Barrett, C., Gonzalez, J. E., Liang, P., Ré, C., Stoica, I., & Zhang, C. (2023). *FlexGen: High-throughput generative inference of large language models with a single GPU* [Preprint]. arXiv.
<https://arxiv.org/abs/2303.06865>